C# Console – Nagy gyakorlati jegyzet

Részletes tételvázlat, bemásolható kódminták, feladatmegoldó sablonok

• változók, típusok, operátorok
• elágazások, ciklusok, metódusok
• tömbök, listák, stringek, fájlkezelés
• OOP, kivételkezelés, LINQ
• gyakori algoritmusminták és vizsgafeladat sablonok

Megjegyzés: a dokumentum nem a teljes C# nyelv minden létező részletét tartalmazza, hanem egy nagyon széles, feladatmegoldásra optimalizált, gyakorlati nagyjegyzetet.

Tartalomjegyzék

1. 1. Alapok és projektfelépítés
2. 2. Változók, típusok, operátorok
3. 3. Be- és kimenet
4. 4. Elágazások és ciklusok
5. 5. Metódusok és paraméterátadás
6. 6. Tömbök, listák, szótárak
7. 7. String kezelés
8. 8. OOP alapok
9. 9. Kivételkezelés
10. 10. Fájlkezelés
11. 11. LINQ alapok
12. 12. Gyakori algoritmusminták
13. 13. Vizsgafeladat sablonok
14. 14. Haladó elemek

1. Alapok és projektfelépítés

A konzolos programozás kiváló terep algoritmusok, logika, input/output és adatfeldolgozás gyakorlására.

Téma	Lényeg
namespace	Névteret jelöl, csoportosítja az osztályokat.
class Program	A program fő osztálya.
static void Main	A belépési pont.
using	Névterek importálása, pl. System, System.IO, System.Linq.

Minimális konzolos program

```
using System;

namespace ElsoProgram
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Helló C#");
        }
    }
}
```

Vizsgán mindig indulj egy tiszta szerkezettel: using-ok, namespace, Program osztály, Main metódus.

2. Változók, típusok, operátorok

- Egész típusok: byte, short, int, long
- Lebegőpontos: float, double, decimal
- Logikai: bool
- Karakter és szöveg: char, string
- Típuskonverzió: implicit és explicit cast, Convert, Parse, TryParse

Alapváltozók és operátorok

```
int kor = 25;
double ar = 1499.99;
string nev = "Balázs";
bool aktiv = true;
char jel = 'A';

int a = 10;
int b = 3;
Console.WriteLine(a + b);   // 13
Console.WriteLine(a - b);   // 7
Console.WriteLine(a * b);   // 30
Console.WriteLine(a / b);   // 3
Console.WriteLine(a % b);   // 1
```

Biztonságos számbeolvasás

```
Console.Write("Adj meg egy számot: ");
string? szoveg = Console.ReadLine();

if (int.TryParse(szoveg, out int szam))
{
    Console.WriteLine($"Duplája: {szam * 2}");
}
else
{
    Console.WriteLine("Hibás számformátum.");
}
```

3. Be- és kimenet

Be- és kimenet alapok

```
Console.Write("Név: ");
string? nev = Console.ReadLine();
Console.WriteLine("Szia " + nev + "!");

int eletkor = int.Parse(Console.ReadLine()!);
Console.WriteLine($"5 év múlva {eletkor + 5} éves leszel.");
```

A ReadLine() stringet ad vissza, ezért a legtöbb numerikus feladatnál Parse/TryParse kell.

4. Elágazások és ciklusok

if – else if – else

```
int pont = 74;

if (pont >= 90)
    Console.WriteLine("jeles");
else if (pont >= 75)
    Console.WriteLine("jó");
else if (pont >= 60)
    Console.WriteLine("közepes");
else
    Console.WriteLine("elégtelen");
```

switch expression

```
int nap = 3;
string nev = nap switch
{
    1 => "hétfő",
    2 => "kedd",
    3 => "szerda",
    _ => "ismeretlen"
};
```

Ciklusminták

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
```

```
}

int j = 0;
while (j < 5)
{
    Console.WriteLine(j);
    j++;
}

do
{
    Console.WriteLine("Egyszer biztos lefut");
} while (false);
```

Tipikus összegzéses feladat

```
int osszeg = 0;
for (int i = 1; i <= 100; i++)
{
    if (i % 2 == 0)
        osszeg += i;
}
Console.WriteLine($"Páros számok összege: {osszeg}");
```

5. Metódusok és paraméterátadás

Metódusok

```
static int Negyzet(int x)
{
    return x * x;
}

static void Koszont(string nev)
{
    Console.WriteLine($"Szia, {nev}!");
}

static void Main(string[] args)
{
    int eredmeny = Negyzet(6);
    Koszont("Anna");
    Console.WriteLine(eredmeny);
}
```

ref és out használat

```
static void Novel(ref int ertek)
{
    ertek++;
}

static void Kiir(out int a, out int b)
{
    a = 10;
    b = 20;
}
```

Feladatoknál szervezettebb a program, ha minden külön részfeladatot külön metódusba szervezel.

6. Tömbök, listák, szótárak

Tömbök

```
int[] tomb = { 5, 2, 8, 1, 9 };
Console.WriteLine(tomb[0]);
Console.WriteLine(tomb.Length);

for (int i = 0; i < tomb.Length; i++)
{
    Console.WriteLine(tomb[i]);
}
```

Lista használata

```
List<string> nevek = new List<string>();
nevek.Add("Anna");
nevek.Add("Béla");
nevek.Remove("Anna");

foreach (string nev in nevek)
{
    Console.WriteLine(nev);
}
```

Szótár használata

```
Dictionary<string, int> pontok = new Dictionary<string, int>();
pontok["Anna"] = 92;
pontok["Béla"] = 81;

if (pontok.ContainsKey("Anna"))
{
    Console.WriteLine(pontok["Anna"]);
}
```

Kollekció	Mikor használd
Tömb	Fix méret, gyors indexelés.
List<T>	Dinamikus méret, gyakori általános megoldás.
Dictionary<TKey,TValue>	Kulcs-érték párok, gyors keresés kulcs szerint.

7. String kezelés

String műveletek

```
string szoveg = "almafa";
Console.WriteLine(szoveg.Length);
Console.WriteLine(szoveg.ToUpper());
Console.WriteLine(szoveg.Contains("fa"));
Console.WriteLine(szoveg.Substring(0, 4));
```

Split használata

```
string adat = "kutya;12;barna";
string[] reszek = adat.Split(';');
```

```
string allat = reszek[0];
int kor = int.Parse(reszek[1]);
string szin = reszek[2];
```

Trim és Replace

```
string mondat = "   C# vizsga   ";
string tiszta = mondat.Trim();
string csere = tiszta.Replace("vizsga", "gyakorlat");
```

8. OOP alapok

Egyszerű osztály

```
class Auto
{
    public string Rendszam { get; set; }
    public int Sebesseg { get; set; }

    public Auto(string rendszam, int sebesseg)
    {
        Rendszam = rendszam;
        Sebesseg = sebesseg;
    }

    public void Kiir()
    {
        Console.WriteLine($"{Rendszam} - {Sebesseg} km/h");
    }
}
```

Objektumpéldány létrehozása

```
Auto auto1 = new Auto("ABC-123", 90);
auto1.Kiir();
```

Öröklődés és felüldefiniálás

```
class Jarmu
{
    public virtual void HangotAd()
    {
        Console.WriteLine("Általános járműhang");
    }
}

class Motor : Jarmu
{
    public override void HangotAd()
    {
        Console.WriteLine("Brumm-brumm");
    }
}
```

OOP-s feladatnál tipikusan kérnek adattagokat, konstruktort, property-ket, metódusokat, öröklődést vagy felülírást.

9. Kivételkezelés

try-catch-finally

```
try
{
    int a = 10;
    int b = 0;
    int c = a / b;
}
catch (DivideByZeroException)
{
    Console.WriteLine("Nullával nem lehet osztani.");
}
catch (Exception ex)
{
    Console.WriteLine("Hiba: " + ex.Message);
}
finally
{
    Console.WriteLine("A művelet lezárult.");
}
```

10. Fájlkezelés

Egyszerű fájlírás és olvasás

```
using System.IO;

File.WriteAllText("adat.txt", "Első sor
Második sor");
string tartalom = File.ReadAllText("adat.txt");
Console.WriteLine(tartalom);
```

Sorokra bontott fájlolvasás

```
string[] sorok = File.ReadAllLines("emberek.txt");
foreach (string sor in sorok)
{
    string[] mezok = sor.Split(';');
    Console.WriteLine($"Név: {mezok[0]}, kor: {mezok[1]}");
}
```

StreamWriter mintakód

```
using (StreamWriter sw = new StreamWriter("hibak.txt", append: true))
{
    sw.WriteLine("Túl nagy sebesség: ABC-123");
}
```

11. LINQ alapok

LINQ alapok

```
using System.Linq;

List<int> szamok = new List<int> { 5, 2, 8, 1, 9, 4 };
```

```
var parosak = szamok.Where(x => x % 2 == 0).ToList();
var rendezve = szamok.OrderBy(x => x).ToList();
int max = szamok.Max();
double atlag = szamok.Average();
```

LINQ objektumokra

```
var emberek = new List<(string Nev, int Kor)>
{
    ("Anna", 22),
    ("Béla", 30),
    ("Cecil", 19)
};

var felnottek = emberek.Where(e => e.Kor >= 18)
                       .OrderBy(e => e.Nev)
                       .ToList();
```

12. Gyakori algoritmusminták

Minimumkeresés

```
// Minimumkeresés
int min = tomb[0];
for (int i = 1; i < tomb.Length; i++)
{
    if (tomb[i] < min)
        min = tomb[i];
}
```

Megszámlálás

```
// Megszámlálás
int db = 0;
foreach (int x in tomb)
{
    if (x > 10)
        db++;
}
```

Eldöntés

```
// Eldöntés
bool vanNegativ = false;
foreach (int x in tomb)
{
    if (x < 0)
    {
        vanNegativ = true;
        break;
    }
}
```

Kiválogatás

```
// Kiválogatás
List<int> pozitivak = new List<int>();
foreach (int x in tomb)
{
    if (x > 0)
```

```
        pozitivak.Add(x);
}
```

Rendezés kézzel

```
// Buborékrendezés
for (int i = 0; i < tomb.Length - 1; i++)
{
    for (int j = 0; j < tomb.Length - 1 - i; j++)
    {
        if (tomb[j] > tomb[j + 1])
        {
            int temp = tomb[j];
            tomb[j] = tomb[j + 1];
            tomb[j + 1] = temp;
        }
    }
}
```

13. Vizsgafeladat sablonok

Szöveges állomány feldolgozása

```
string[] sorok = File.ReadAllLines("adatok.txt");
int osszeg = 0;
int db = 0;

foreach (string sor in sorok)
{
    if (int.TryParse(sor, out int ertek))
    {
        osszeg += ertek;
        db++;
    }
}

if (db > 0)
    Console.WriteLine($"Átlag: {(double)osszeg / db}");
```

Objektumlista feltöltése fájlból

```
class Tanulo
{
    public string Nev { get; set; }
    public int Pont { get; set; }

    public Tanulo(string nev, int pont)
    {
        Nev = nev;
        Pont = pont;
    }
}

List<Tanulo> tanulok = new List<Tanulo>();
string[] sorok = File.ReadAllLines("pontok.txt");

foreach (string sor in sorok)
{
    string[] m = sor.Split(';');
```

```
    tanulok.Add(new Tanulo(m[0], int.Parse(m[1])));
}
```

Menüs konzolos alkalmazás

```
bool kilep = false;
while (!kilep)
{
    Console.WriteLine("1 - Hozzáadás");
    Console.WriteLine("2 - Listázás");
    Console.WriteLine("0 - Kilépés");
    Console.Write("Választás: ");
    string? valasz = Console.ReadLine();

    switch (valasz)
    {
        case "1":
            Console.WriteLine("Hozzáadás...");
            break;
        case "2":
            Console.WriteLine("Listázás...");
            break;
        case "0":
            kilep = true;
            break;
        default:
            Console.WriteLine("Ismeretlen menüpont.");
            break;
    }
}
```

14. Haladó elemek

Modern C# elemekből ízelítő

```
// enum
public enum Allapot
{
    Uj,
    FeldolgozasAlatt,
    Kesz
}

// rekord (modernebb adattárolásra)
public record Termek(string Nev, decimal Ar);

// nullable kezelése
string? bemenet = Console.ReadLine();
if (!string.IsNullOrWhiteSpace(bemenet))
{
    Console.WriteLine(bemenet);
}
```

async / await alap

```
// aszinkron konzolos példa
using System.Net.Http;

static async Task Main(string[] args)
```

```
{
    using HttpClient client = new HttpClient();
    string html = await client.GetStringAsync("https://example.com");
    Console.WriteLine(html.Substring(0, 100));
}
```

Haladó részeknél elég, ha felismered a mintát és tudod, mikor érdemes használni őket.